

Reconhecimento Automático de Placas Veiculares Brasileiras (ALPR Mercosul)

Disciplina: Processamento e Análise de Imagens — Pós-Graduação **Autor:** Raphael Magalhães · **Data:** setembro de 2026 · **Repositório:** `alpr-mercotel`

Todos os números deste relatório foram produzidos por código executado e estão gravados em `resultados/tabelas/`. Nenhum valor foi estimado.

1. Introdução

ALPR (*Automatic License Plate Recognition*) é a tarefa de, a partir da fotografia de um veículo, devolver o texto da sua placa. Tem aplicação direta em controle de acesso, pedágio automático, fiscalização e inventário de frota.

Este trabalho constrói o pipeline completo — da foto ao texto — em seis etapas (detecção, recorte, pré-processamento, localização dos caracteres, classificação e validação por formato) e o mede de ponta a ponta contra um gabarito transcrito por humano.

O resultado principal não é uma métrica, é um **diagnóstico**: as métricas por etapa eram boas (0,992 de mAP na detecção, 0,9435 de acurácia na CNN) **enquanto o sistema completo lia zero placas corretamente**. A investigação desse zero mostrou que o gargalo estava numa única etapa — a segmentação dos caracteres — e substituí-la dobrou a acurácia por caractere e tirou a acurácia por placa do zero.

2. O problema

2.1 A tarefa

Dada a foto de um veículo, produzir a string de 7 caracteres da placa. A dificuldade não está em "ler texto": está em que a placa é um objeto **minúsculo** dentro de uma cena complexa — área mediana de **1,26%** da imagem no treino e **0,70%** no teste.

2.2 Os dois formatos brasileiros

O Brasil convive com dois padrões, e essa convivência é central para o trabalho:

Padrão	Máscara	Exemplo	Marca visual
Mercosul (desde 2018)	3 letras · 1 dígito · 1 letra · 2 dígitos (LLLDLDD)	ABC1D23	tarja azul com "BRASIL"
Brasileiro antigo	3 letras · 4 dígitos (LLLDDDD)	ABC1234	tarja com estado e cidade

Ambos têm 7 caracteres, mas **a tipagem de cada posição difere**. Isso é conhecimento de domínio explorável: sabendo o layout, a quinta posição de uma placa Mercosul *só pode* ser letra. Um `0` previsto ali é corrigível sem treinar nada a mais.

2.3 As bases de dados

Duas bases públicas do Roboflow Universe, porque as duas etapas exigem anotações incompatíveis — detecção precisa de veículos com a placa demarcada; reconhecimento precisa de cada caractere demarcado.

Detecção — `trafficbr/vehicle-plate-color v2`, classe única `plate`, imagens 416×416:

Split	Imagens	Caixas	Área mediana da placa
train	12.780	13.386	1,26%
valid	960	995	0,89%
test	257	268	0,70%

Caracteres — `project-swcsj/license-plate-character-extraction v2` (licença MIT), com cada caractere anotado individualmente: **31.718 caracteres válidos** em 36 classes (train 30.530 · valid 958 · test 230), mais 1.752 anotações descartadas por não serem caractere de placa (`EUR`, a tarja europeia, e o hífen `-`).

2.4 Metas

Meta	Alvo
mAP@0.5 na detecção	> 0,90
Acurácia por caractere	> 0,95
Acurácia por placa (os 7 corretos)	> 0,80

3. Dificuldades

3.1 Nenhuma base traz o texto da placa

As duas bases anotam *caixas*, não *texto* — exatamente o que o sistema deve produzir. Sem isso não existe acurácia por placa. Foi construído um conjunto próprio: **30 placas brasileiras (16 Mercosul, 14 do formato antigo) com o texto transcrito e conferido por humano**.

Um cuidado metodológico foi decisivo: em 6 das 30 placas, distinguir 0 de Ø só era possível *aplicando a regra de formato* — mas o sistema avaliado também usa essa regra. Um gabarito assim seria **circular** e inflaria a acurácia. As 8 transcrições duvidosas foram conferidas independentemente por humano.

3.2 Defeitos nos dados

A postura foi **registrar os defeitos, não contorná-los com heurísticas**:

1. **Anotações grosseiramente erradas na base de detecção.** As 12 maiores caixas rotuladas `plate` ocupam 45–63% da imagem com razão largura/altura de 0,71 a 1,70: **são carros inteiros**. Uma placa real tem razão mediana 2,20. Isso obrigou a filtrar por geometria e, como se verá em 5.1, reinterpreta a análise de erros da detecção.
2. **Caracteres deformados na origem.** A base de caracteres foi pré-processada com "*Resize to 640×640 (Stretch)*" — cada imagem esticada sem preservar proporção. A distorção não é uniforme e a proporção original não é recuperável.
3. **Ruído de rótulo.** A inspeção visual encontrou ao menos um caractere rotulado 0 que mostra um H. Não houve filtragem manual de ~31 mil recortes; o ruído é assumido como fonte honesta de erro herdada pelo classificador.
4. **Desbalanceamento severo.** Q tem 120 exemplos e 0 tem 228, contra 400–2.100 das demais. Como ambos são visualmente próximos do dígito Ø, isso foi apontado como risco **antes** do treino — e a previsão se confirmou (seção 5.2).

3.3 O gargalo: a segmentação clássica tem teto

Medido o pipeline completo pela primeira vez, a acurácia por placa foi **zero**. A investigação encontrou dois problemas de natureza bem diferente.

Um erro de recorte. A constante `corte_superior = 0,35`, criada para descartar a tarja "BRASIL", estava sendo aplicada **mesmo quando a entrada já era apenas a faixa dos caracteres**. Numa imagem de 200×62 px, isso começa a segmentar na linha 21 quando os caracteres começam por volta da linha 6: **o terço superior de cada caractere era descartado antes da classificação**. Isso explica as previsões anteriores cheias de I, 1, 4, L e P — as formas que sobram de um caractere sem topo.

Variante (ablação)	corte_superior	Acc. caractere	Acc. placa
baseline	0,35	0,0612	0,000
só corte_superior = 0	0,00	0,6122	0,2143
só aperto vertical	0,35	0,1224	0,000
corte = 0 + aperto + folga	0,00	0,6122	0,2143

Um único parâmetro respondeu por praticamente todo o ganho — 0,061 para 0,612.

Um limite do método. Mesmo corrigido o recorte, a segmentação por componentes conectados encontra **exatamente 7 blobs em apenas 4 de 14 placas**. Elevar a resolução melhora e satura:

Resolução intermediária	Placas com exatamente 7 blobs
200×62 (original)	4 de 14
400×124	6 de 14
600×186	8 de 14
com abertura morfológica	praticamente sem efeito

Depois da binarização, os caracteres se encostam uns nos outros e na moldura: um componente conectado **não corresponde** a um caractere. **É o teto do método, não um parâmetro mal ajustado** — nenhum ajuste torna isso. Foi o que motivou a solução descrita a seguir.

4. Solução proposta

```

foto do veículo
└─> [1] YOLO11n detecta a placa
    └─> [2] recorte com margem de 8% + endireitamento
        └─> [3] localização dos 7 caracteres
            └─ (A) clássica: cinza + CLAHE + Otsu + componentes conectados
            └─ (C) aprendida: YOLO11n detector de caracteres
                └─> [4] classificação de cada caractere
                    └─ CNN de 36 classes, ou
                    └─ a própria classe dada pelo detector
                        └─> [5] regra do formato (máscara por layout)
                            └─> "ABC1D23"

```

A etapa [3] foi implementada de **duas formas independentes**, e a comparação entre elas é o experimento central (seção 5.4).

[1] Detecção. YOLO11n treinado por 40 épocas em GPU T4 do Colab. O modelo mais leve da família foi escolhido deliberadamente: o objeto é único, de uma classe e de geometria muito regular. Sai a caixa de maior confiança, com margem de 8% para não cortar caracteres na borda.

[2] Endireitamento. `endireitar()` estima a inclinação via `cv2.minAreaRect` sobre o maior contorno e aplica rotação afim. Corrige rotação *no plano*, não perspectiva — a correção adequada exigiria os 4 cantos da placa, que nenhuma base anota.

[3-A] Segmentação clássica. Escala de cinza → **CLAHE** (`clipLimit=2.0`, `tileGridSize=(8,8)`) → **Otsu** invertido → componentes conectados, com junção e divisão por vales da projeção vertical até fechar 7 blocos → normalização para 32×32 . O CLAHE foi preferido à equalização global porque a placa costuma ter iluminação desigual (sombra de um lado, reflexo do outro); a global amplificaria o ruído do fundo.

[3-C] Detecção aprendida de caracteres. Um segundo YOLO11n treinado sobre as **32.225 caixas de caractere** já anotadas, com 38 classes. Recebe o recorte da placa e devolve a caixa e a classe de cada caractere — sem binarização, sem componentes conectados, sem supor que os caracteres estejam separados.

O limiar usado é **conf = 0,05**, muito abaixo do usual 0,25. A escolha é medida (seção 5.5): como a placa tem exatamente 7 caracteres e o sistema fica com os 7 mais confiantes, um falso positivo é barato — o caro é *faltar* caractere, que estraga a leitura inteira e ainda invalida a regra de formato.

[4] CNN de caracteres. Rede pequena treinada do zero, **359.588 parâmetros**: 3 blocos `Conv2D` + `MaxPooling`, `Dense(128)`, `Dropout(0.3)` e `softmax` de 36 classes. Entrada $32 \times 32 \times 1$ binarizada.

[5] Regra do formato. O layout é detectado pela **tarja azul** (fração de pixels azuis em HSV), método que não depende do acerto do classificador. Com o layout, `aplicar_mascara()` troca caracteres impossíveis pelo equivalente visual:

```

posição exige LETRA → 0→0  1→I  2→Z  4→A  5→S  6→G  7→T  8→B
posição exige DÍGITO → 0→0  Q→0  D→0  I→1  L→1  Z→2  A→4  S→5  G→6  T→7  B→8

```

São 15 linhas derivadas da legislação, com zero parâmetros treinados.

Métricas. mAP@0.5 e mAP@0.5:0.95 na detecção (IoU implementado do zero e testado); **acurácia por caractere** (fração das 210 posições corretas); **acurácia por placa** (as 7 corretas — é o que importa na prática: 6 de 7 não abre a cancela); **erros por posição**, para separar falha de localização de confusão de classe.

5. Resultados

5.1 Detecção da placa (teste: 257 imagens)

Métrica	Valor
mAP@0.5	0,992
mAP@0.5:0.95	0,8344
Precisão / Recall	0,9776 / 0,9768

Meta superada com folga. A distância entre 0,992 e 0,8344 é informativa: o detector **encontra** quase todas as placas, mas o ajuste fino da caixa é menos preciso — o que é aceitável, já que a etapa seguinte aplica margem de 8% de qualquer modo.

Apenas 2 das 257 placas não foram detectadas. Os piores casos por IoU (0,000 a 0,021) foram inicialmente li-dos como falha do detector; a investigação das anotações (seção 3.2) mostrou que **boa parte são anotações erradas da base**, em que a referência marca o veículo inteiro. **O IoU é baixo porque o gabarito está errado, não porque o detector falhou** — o desempenho real é melhor do que aquelas linhas sugerem.

5.2 CNN de caracteres — o efeito do balanceamento

Rodada	Ajuste	Acurácia (teste)	Épocas	Q e O
1ª	sem peso de classe	0,9348	13	0% de recall nas duas
2ª	class_weight balanceado	0,9435	26	O: 100% · Q: 33%

A previsão feita na análise dos dados se confirmou exatamente: **as duas classes mais raras do treino foram as duas que o modelo sem balanceamento ignorou por completo**. O balanceamento (`peso = total / (36 × contagem)`), uma linha no `fit()` resolveu Q e O, mas cobrou preço: o recall do dígito `0` caiu para 0,56, porque a classe `0` passou a "roubar" exemplos dele. Não é resultado limpo — é deslocamento do erro.

Limitação honesta: o teste tem apenas **230 caracteres para 36 classes** — `Q` aparece 3 vezes e `0` apenas 1. Não é possível afirmar se o recall de 33% em `Q` é representativo ou ruído. A decisão foi **parar de ajustar hiperparâmetros**, porque seguir seria otimizar sobre ruído de 1 a 3 exemplos.

5.3 Detector de caracteres, isoladamente

Split	Imagens	mAP@0.5	mAP@0.5:0.95	Precisão	Recall
valid	144	0,8678	0,6395	0,8733	0,8037
test	36	0,9305	0,7029	0,9284	0,8621

O recall de 0,862 no teste é o número a reter: **ele ainda perde ~14% dos caracteres**, e é daí que vem boa parte do erro remanescente do sistema.

5.4 O experimento central — as mesmas 30 placas

Pipeline	Acc. caractere (sem regra)	Acc. caractere (com regra)	Acc. placa (sem regra)	Acc. placa (com regra)
A) clássico (segmentar) + CNN	0,2714	0,3000	0/30	0/30
B) YOLO só para as caixas + a mesma CNN	0,5476	0,6190	2/30	7/30 (0,2333)
C) YOLO dá caixa e classe	0,6190	0,7190	4/30	10/30 (0,3333)

O experimento B é o que prova a tese. Mantendo a CNN *idêntica* dos dois lados e trocando **apenas** a forma de localizar os caracteres, a acurácia por caractere **dobra** ($0,300 \rightarrow 0,619$) e a acurácia por placa sai de zero para 7 em 30. O gargalo era a **segmentação** — não a CNN, não a base, não o detector de placas. O passo C, que troca também o classificador, acrescenta mais 10 pontos.

5.5 A regra de formato e o limiar de confiança

A regra rendeu **+10,0 pontos** de acurácia por caractere no pipeline C ($0,619 \rightarrow 0,719$), contra apenas **+2,9** no pipeline A. A leitura é direta: **a regra de formato só conserta leitura que já está quase certa**. Sobre uma segmentação ruim ela não tem o que corrigir — troca um caractere errado por outro caractere errado do tipo certo.

Varredura do limiar do detector de caracteres:

conf	Placas com 7 caracteres	Acc. caractere	Acc. placa
0,05	29/30	0,7190	0,3333
0,10	27/30	0,6952	0,3000
0,25 (padrão)	23/30	0,6571	0,2333
0,50	8/30	0,4810	0,1667

O padrão da biblioteca (0,25) custaria 7 pontos de acurácia por caractere e 10 de acurácia por placa.

5.6 Erros por posição (de 30 placas, com regra)

Pipeline	pos1	pos2	pos3	pos4	pos5	pos6	pos7	Total
A) clássico	24	18	21	17	21	23	23	147
B) YOLO caixas + CNN	10	9	13	13	13	12	10	80
C) YOLO caixa+classe	5	9	9	11	10	11	4	59

Dois padrões saltam. A queda de A para C é **uniforme** — nenhuma posição resiste, o que confirma que o problema era sistêmico. E no pipeline C **as extremidades erram bem menos que o miolo** (5 e 4, contra 9 a 11 no centro). Se a falha fosse de *localização*, o padrão seria o oposto — as pontas é que se perderiam no recorte. O padrão observado aponta para **confusão de classe** entre caracteres visualmente próximos.

5.7 Confronto com as metas

Meta	Alvo	Obtido	Situação
mAP@0.5 — detecção da placa	> 0,90	0,992	✅ superada
mAP@0.5 — detecção de caracteres	—	0,9305	✅ (meta não definida)
Acurácia por caractere — CNN isolada	> 0,95	0,9435	⚠️ próxima, não atingida
Acurácia por caractere — fim a fim	> 0,95	0,7190	❌ não atingida
Acurácia por placa — fim a fim	> 0,80	0,3333	❌ não atingida

Duas metas não foram atingidas, e isso precisa ser dito sem rodeio: o sistema lê corretamente 10 das 30 placas, distante dos 80% projetados.

6. Conclusão

6.1 Por que a acurácia por placa é tão menor que a por caractere

Com acurácia por caractere p e erros independentes, a acurácia por placa esperada seria p^7 :

- com $p = 0,7190 \rightarrow p^7 = 0,099$, isto é, ~ 3 placas em 30;
- o observado é 10 em 30 (0,333) — mais de três vezes melhor.

A diferença revela que **os erros não são independentes**: concentram-se em placas ruins (baixa resolução, reflexo, ângulo), enquanto as placas boas saem inteiramente corretas. O sistema não erra um caractere aqui e outro ali — acerta a placa toda ou erra várias posições da mesma placa. Operacionalmente isso é boa notícia: um filtro de qualidade de imagem na entrada tende a separar bem os dois regimes.

6.2 O que o trabalho mostra

1. **Detectar a placa é o problema fácil; localizar os caracteres é o difícil.** A detecção chegou a 0,992 de mAP@0.5 com um modelo leve. A segmentação clássica encontrou os 7 caracteres em menos de um terço das placas.
2. **O limite era do método, não do ajuste.** Elevar a resolução de 200×62 para 600×186 levou o acerto de 4/14 para 8/14 e saturou. Depois da binarização os caracteres se tocam e deixam de existir como componentes separados.
3. **Trocar a etapa gargalo valeu mais que otimizar as demais.** O experimento controlado — mesma CNN, segmentação diferente — dobrou a acurácia por caractere. Identificar corretamente qual etapa limita o sistema valeu mais que qualquer ajuste de hiperparâmetro.
4. **Conhecimento de domínio complementa o modelo, mas não o substitui.** A regra de formato rendeu +10 pontos sobre um reconhecimento bom e apenas +2,9 sobre um ruim: corrige erro de *confusão*, não erro de *percepção*.
5. **Um pipeline só pode ser otimizado depois de medido de ponta a ponta.** As métricas por etapa eram boas enquanto o sistema completo lia zero placas.

6.3 Limitações

1. **As metas de reconhecimento não foram atingidas** — 0,719 por caractere (meta 0,95) e 0,333 por placa (meta 0,80).
2. **O conjunto de avaliação é pequeno**: 30 placas, cada uma valendo 3,3 pontos percentuais. Insuficiente para distinguir com confiança 0,33 de 0,45.
3. **O detector de caracteres não completou o treino planejado** — a sessão gratuita do Colab caiu por volta da época 30 de 40. Há margem de melhora não medida.
4. **A detecção de layout acerta 24 de 30.** Cada erro aplica a máscara errada e corrompe a correção justamente onde ela deveria ajudar.
5. **Perspectiva não é corrigida**, apenas rotação no plano; e **placas de motocicleta, em duas linhas, estão fora do escopo**.
6. **Condições noturnas não foram avaliadas** — as bases são majoritariamente diurnas.

7. O limiar de rejeição (0,70) não foi recalibrado. Como a confiança mínima observada fica entre 0,059 e 0,759 (mediana 0,344), o sistema marca quase tudo como `revisao_manual`: conservador e seguro, mas ainda pouco útil.

Anexo — Reprodutibilidade

Treinos em GPU T4 do Google Colab; toda a avaliação e inferência em CPU local (Apple M1). Os módulos de `src/` são cobertos por **28 testes** em `pytest`, e os notebooks importam deles em vez de redefinir funções — o código medido é o código entregue.

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest tests/ -v                                # 28 testes das funções puras

python notebooks/07_comparar_pipeline.py        # comparação dos 3 pipelines
python notebooks/08_varredura_conf_e_erros.py    # limiar e erros por posição
python notebooks/09_avaliar_yolo_caracteres.py   # mAP do detector de caracteres
```

Os notebooks `01_` a `06_` reproduzem, na ordem, a preparação dos dados, o treino do detector, a geração da base de caracteres, o treino da CNN, o pipeline fim a fim e o treino do detector de caracteres. Modelos treinados não são versionados por tamanho.

Tabelas em `resultados/tabelas/` (16 arquivos) e **figuras** em `resultados/figuras/` (13 arquivos) — toda métrica citada acima tem origem nesses arquivos.